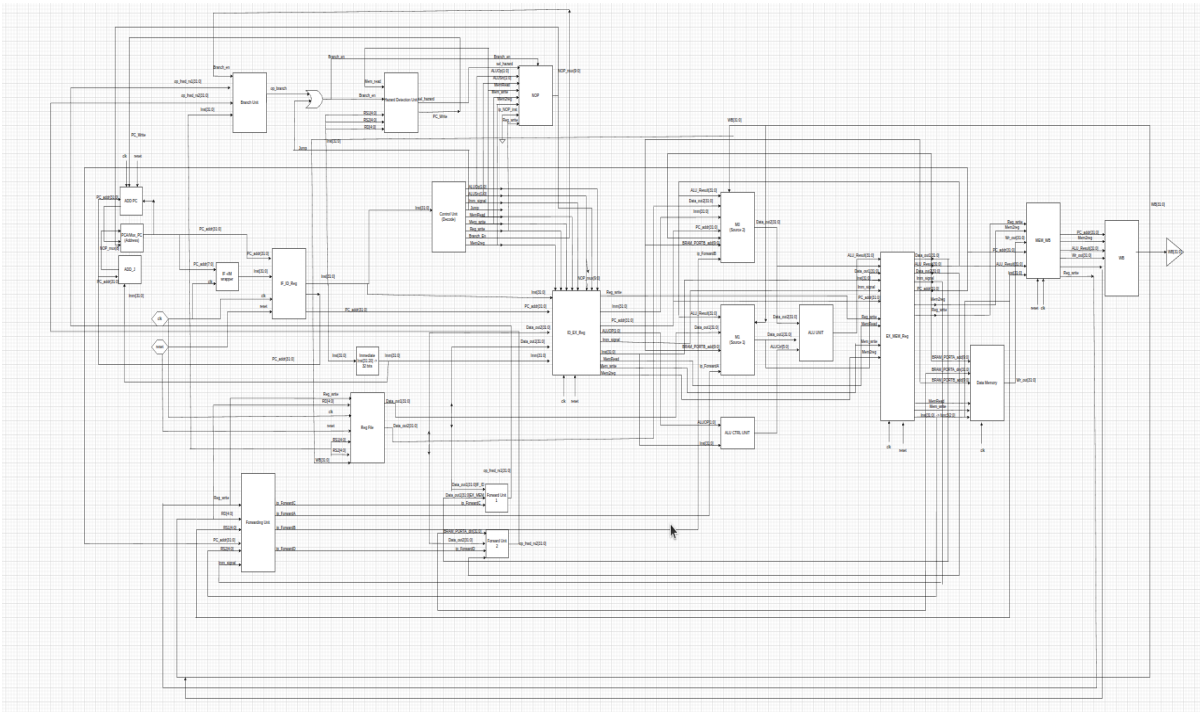
RISC V:

Architecture:



Simulation Results:

R-type Instruction:

SLTU

<u>Instruction:</u>

addi x6, x0, 94 # Load immediate 94 into x6 (t1), since x0 is always 0

addi x7, x0, 8 # Load immediate 8 into x7 (t2), since x0 is always 0

sltu x29, x6, x7 # x29 (t4) = 1 if x6 < x7 (unsigned), else 0

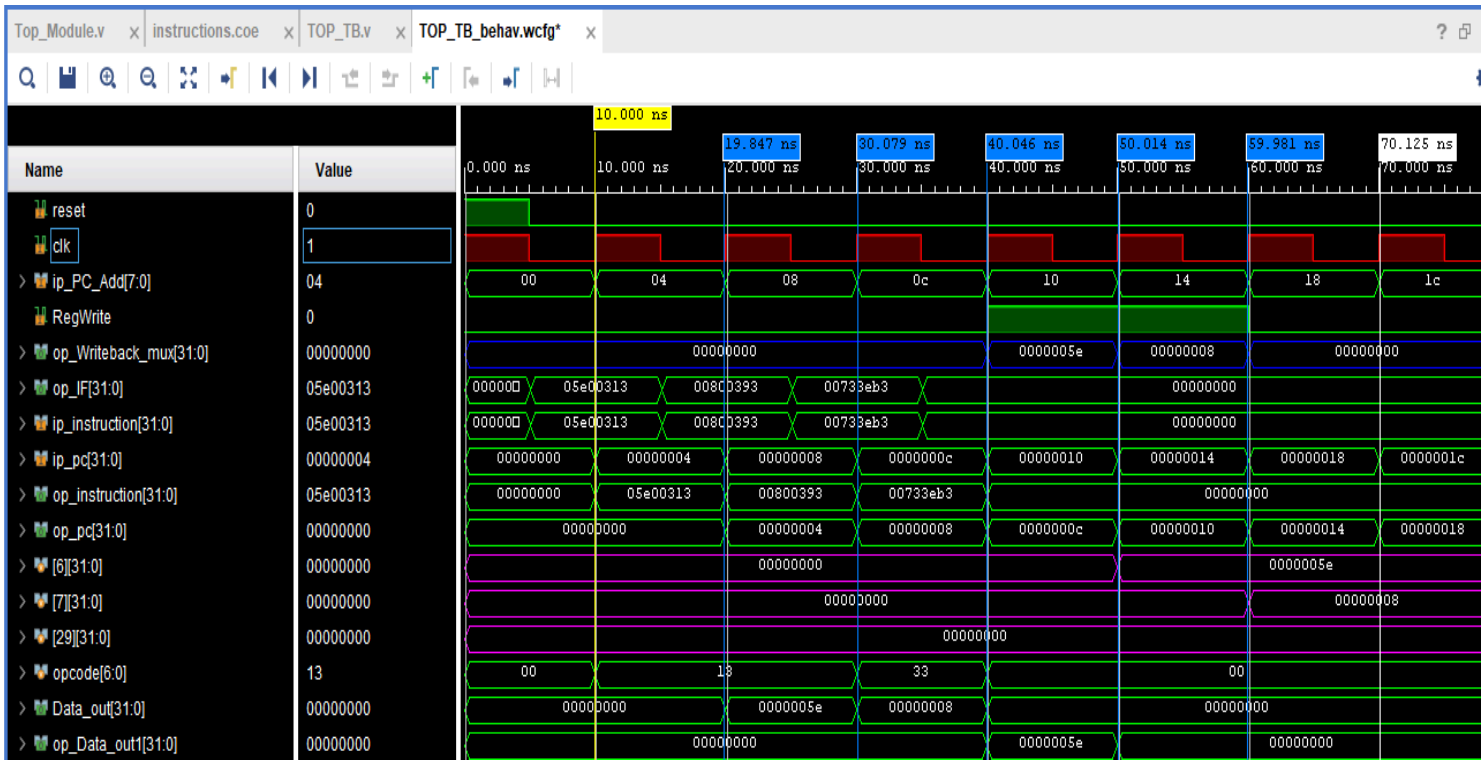
Machine Codes

05E00313,

00800393,

00733EB3;

Observations:



Cycle	Time	Stage	Description	Key Signal Values
C3	20–30 ns	IF	Fetch instruction	ip_pc = 8, ip_instruction = 00733eb3
C4	30–40 ns	ID	Decode instruction	rs1 = x6 = 94, rs2 = x7 = 8, opcode = 33
C5	40–50 ns	EX	ALU unsigned comparison	ALU_Result = (94 < 8) = 0
C6	50–60 ns	MEM	No memory operation	—
C7	60–70 ns	WB	Write back result	x29 = 0, RegWrite = 1

Instruction	Operation	Result	Completion
SLTU	x29 = (x6 < x7) → (94 < 8)	0	70 ns

I-type Instruction:

SLTIU

Instruction:

addi x6, x0, 94 # Load immediate 94 into x6 (t1), since x0 is always 0

sltiu x29, x6, 100 # x29 (t4) = 1 if x6 < 100 (unsigned comparison), else 0

Machine Codes

05E00313;

06433E93;

Observations:

Cycle	Time	Stage	Description	Key Signal Values
C1	0–10 ns	IF	Fetch instruction	ip_pc=0, ip_instruction =05e00313
C2	10–20 ns	ID	Decode	rs1=x0=0, imm=94, opcode=13
C3	20–30 ns	EX	ALU add	ALU_Result= 94, Mux_Data=9 4
C4	30–40 ns	MEM	No memory op	—
C5	40–50 ns	WB	Write back	[6]=94, RegWrite=1

Cycle	Time	Stage	Description	Key Signal Values
C2	10–20 ns	IF	Fetch instruction	ip_pc=4, ip_instruction=06433e93
C3	20–30 ns	ID	Decode	rs1=x6, imm=100
C4	30–40 ns	EX	Unsigned comparison	ALU_Result = (94 < 100) = 1
C5	40–50 ns	MEM	No memory op	—
C6	50–60 ns	WB	Write back	[29]=1, RegWrite=1

Instruction	Operation	Result	Completion
ADDI	x6 = 0 + 94	94	50 ns
SLTIU	x29 = (94 < 100)	1	60 ns

B-type Instruction:

Beq:

<u>Instruction:</u>

addi x5, x0, 8 # x5 = 8

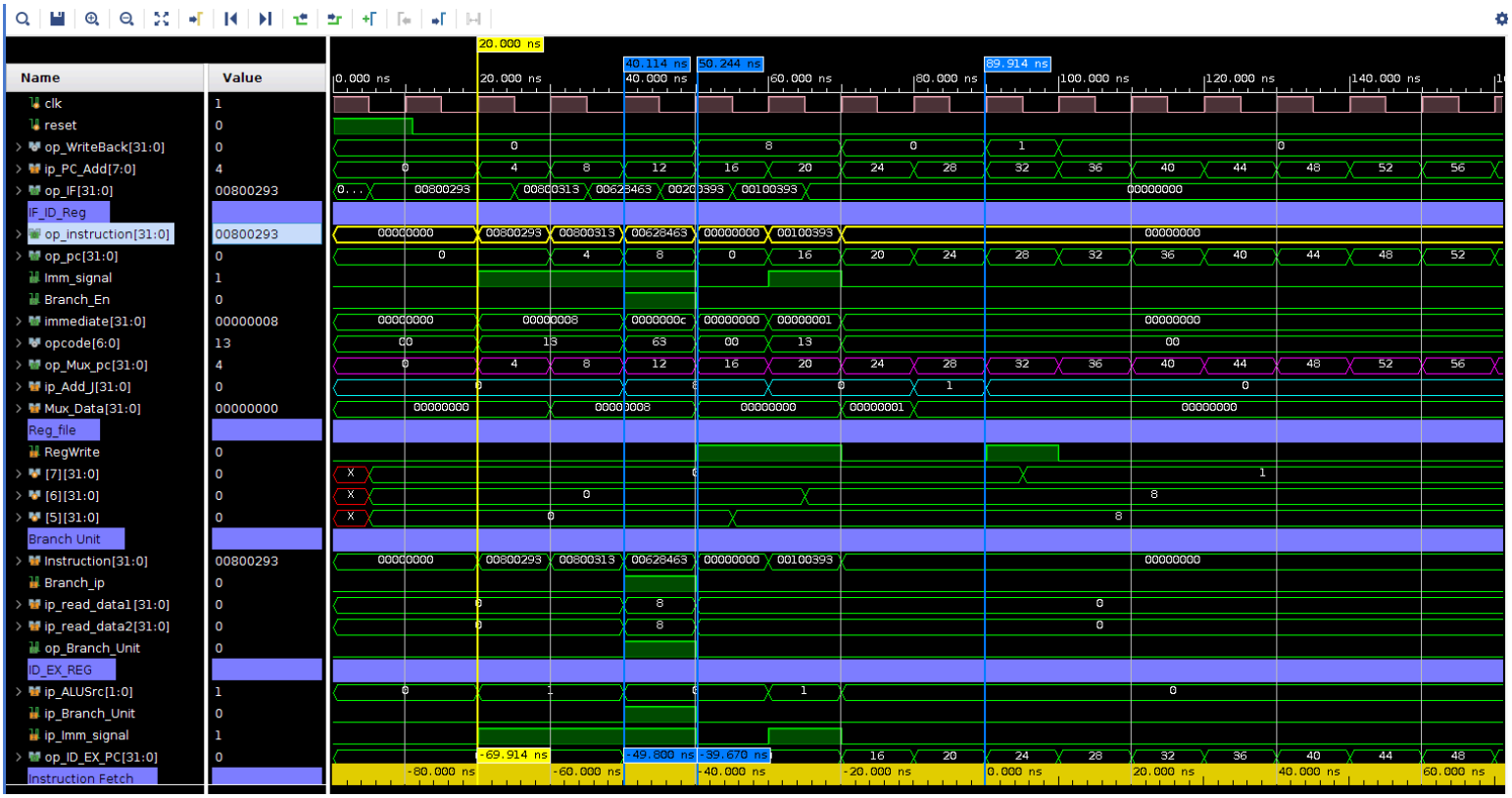
addi x6, x0, 8 # x6 = 8

```
beq  x5, x6, TARGET  # 8 == 8? YES → jump
addi x7, x0, 2       # skipped
TARGET: addi x7, x0, 1  # x7 = 1
```

<u>Machine Code:</u>

```
memory_initialization_radix=16;
memory_initialization_vector=
00800293,
00800313,
00628463,
00200393,
00100393;
```

Observations:



Cycle	Time	Stage	Description	Key Signal Values
C3	20–30 ns	IF	Fetch instruction	ip_pc = 8, ip_instruction = 00628463
C4	30–40 ns	ID	Decode instruction	rs1 = x5 = 0, rs2 = x6 = 8, opcode = 63
C5	40–50 ns	EX	Branch comparison	ALU_Result = (0 == 8) = 0
C6	50–60 ns	MEM	Branch decision	Branch not taken
C7	60–70 ns	WB	No register write	RegWrite = 0

Load-Store Instruction:

Instruction:

addi x4, x0, 5

addi x5, x0,10

sw x5, 0(x2)

lw x1, 0(x2)

add x6,x1,x4

Machine Code:

memory_initialization_radix=16;

memory_initialization_vector=

00500213,

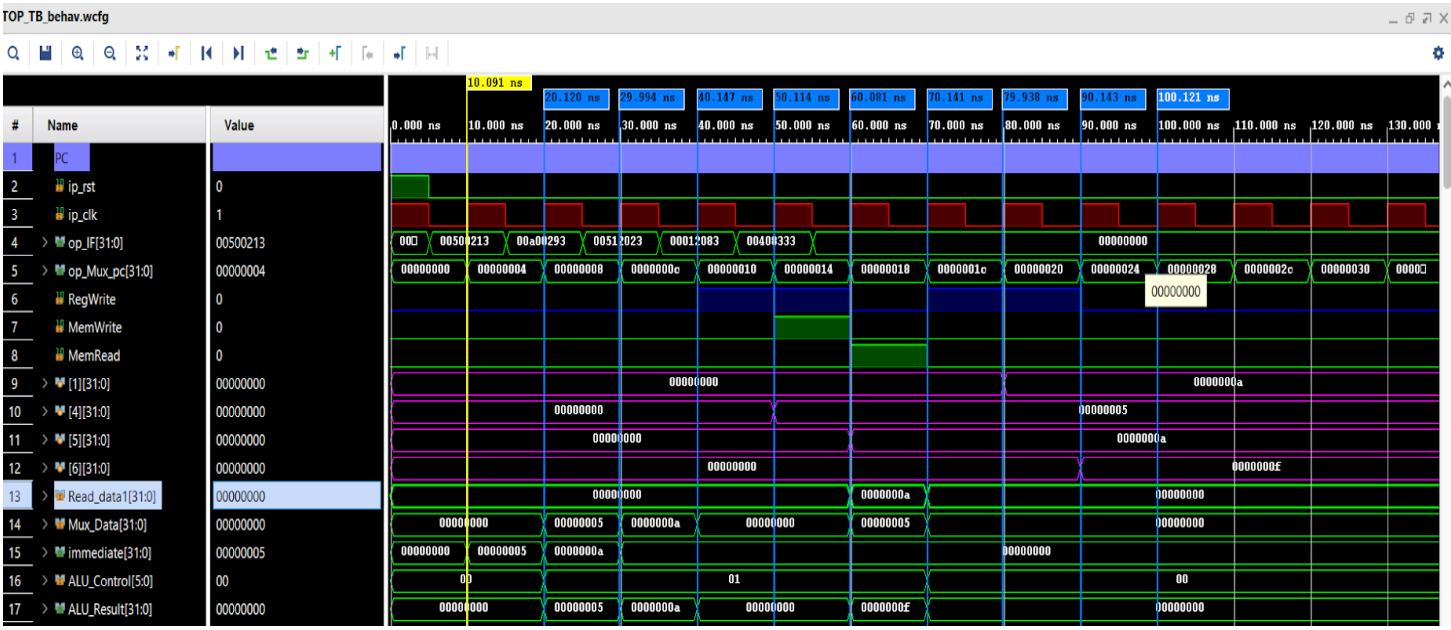
00A00293,

00512023,

00012083,

00408333;

Observations:



ADDI Instruction (`00500213`)

Cycle	Time	Stage	Description	Key Signal Values
C1	0–10 ns	IF	Fetch instruction	ip_pc = 0, ip_instruction = 00500213
C2	10–20 ns	ID	Decode instruction	rs1 = x0 = 0, imm = 5
C3	20–30 ns	EX	ALU addition	ALU_Result = 0 + 5 = 5
C4	30–40 ns	MEM	No memory operation	—
C5	40–50 ns	WB	Write back result	x4 = 5, RegWrite = 1

ADDI Instruction (`00A00293`)

Cycle	Time	Stage	Description	Key Signal Values
C2	10–20 ns	IF	Fetch instruction	ip_pc = 4, ip_instruction = 00A00293
C3	20–30 ns	ID	Decode instruction	rs1 = x0 = 0, imm = 10
C4	30–40 ns	EX	ALU addition	ALU_Result = 0 + 10 = 10
C5	40–50 ns	MEM	No memory operation	—
C6	50–60 ns	WB	Write back result	x5 = 10, RegWrite = 1

STORE Instruction (`00512023`)

Cycle	Time	Stage	Description	Key Signal Values
C3	20–30 ns	IF	Fetch instruction	ip_pc = 8, ip_instruction = 00512023
C4	30–40 ns	ID	Decode instruction	rs1 = x2, rs2 = x5 = 10
C5	40–50 ns	EX	Address calculation	ALU_Result = x2 + 0
C6	50–60 ns	MEM	Store data to memory	Mem[x2] = 10
C7	60–70 ns	WB	No register write	RegWrite = 0

LOAD Instruction (`00012083`)

Cycle	Time	Stage	Description	Key Signal Values
C4	30–40 ns	IF	Fetch instruction	ip_pc = 12, ip_instruction = 00012083
C5	40–50 ns	ID	Decode instruction	rs1 = x2
C6	50–60 ns	EX	Address calculation	ALU_Result = x2 + 0
C7	60–70 ns	MEM	Read memory	Read_data = 10
C8	70–80 ns	WB	Write back result	x1 = 10

ADD Instruction (`00408333`)

Cycle	Time	Stage	Description	Key Signal Values
C5	40–50 ns	IF	Fetch instruction	ip_pc = 16, ip_instruction = 00408333
C6	50–60 ns	ID	Decode instruction	rs1 = x1 = 10, rs2 = x4 = 5
C7	60–70 ns	EX	ALU addition	ALU_Result = 10 + 5 = 15
C8	70–80 ns	MEM	No memory operation	—
C9	80–90 ns	WB	Write back result	x6 = 15

Jump Instruction:

JALR

Instruction:

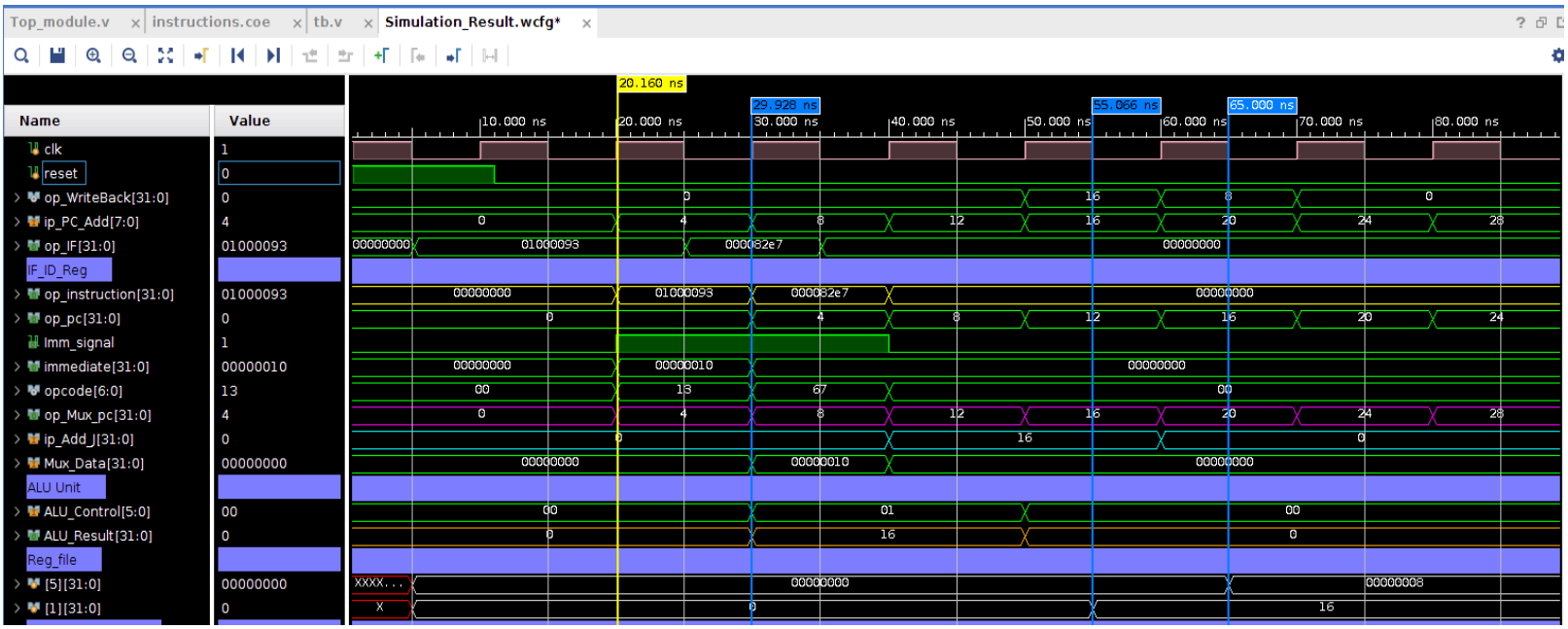
PC	Machine Code	Basic Code	Original Code
	0x0	0x01000093	addi x1 x0 16
	0x4	0x000082E7	jalr x5 x1 0

Machine Code:

01000093,

000082E7;

Observation:



Cycle	Time	Stage	Description	Key Signal Values
C1	0–10 ns	IF	Fetch instruction	ip_pc = 0, ip_instruction = 01000093
C2	10–20 ns	ID	Decode instruction	rs1 = x0 = 0, imm = 16
C3	20–30 ns	EX	ALU addition	ALU_Result = 0 + 16 = 16
C4	30–40 ns	MEM	No memory operation	—
C5	40–50 ns	WB	Write back result	x1 = 16, RegWrite = 1

Cycle	Time	Stage	Description	Key Signal Values
C2	10–20 ns	IF	Fetch instruction	ip_pc = 4, ip_instruction = 000082e7
C3	20–30 ns	ID	Decode instruction	rs1 = x1 = 16, imm = 0
C4	30–40 ns	EX	Target address calculation	Target = x1 + 0 = 16
C5	40–50 ns	MEM	Update PC	PC = 16
C6	50–60 ns	WB	Write return address	x5 = PC + 4

Instruction	Operation	Result	Completion
ADDI	x1 = 0 + 16	16	50 ns
JALR	PC = x1 + 0	Jump to address 16	60 ns

JAL

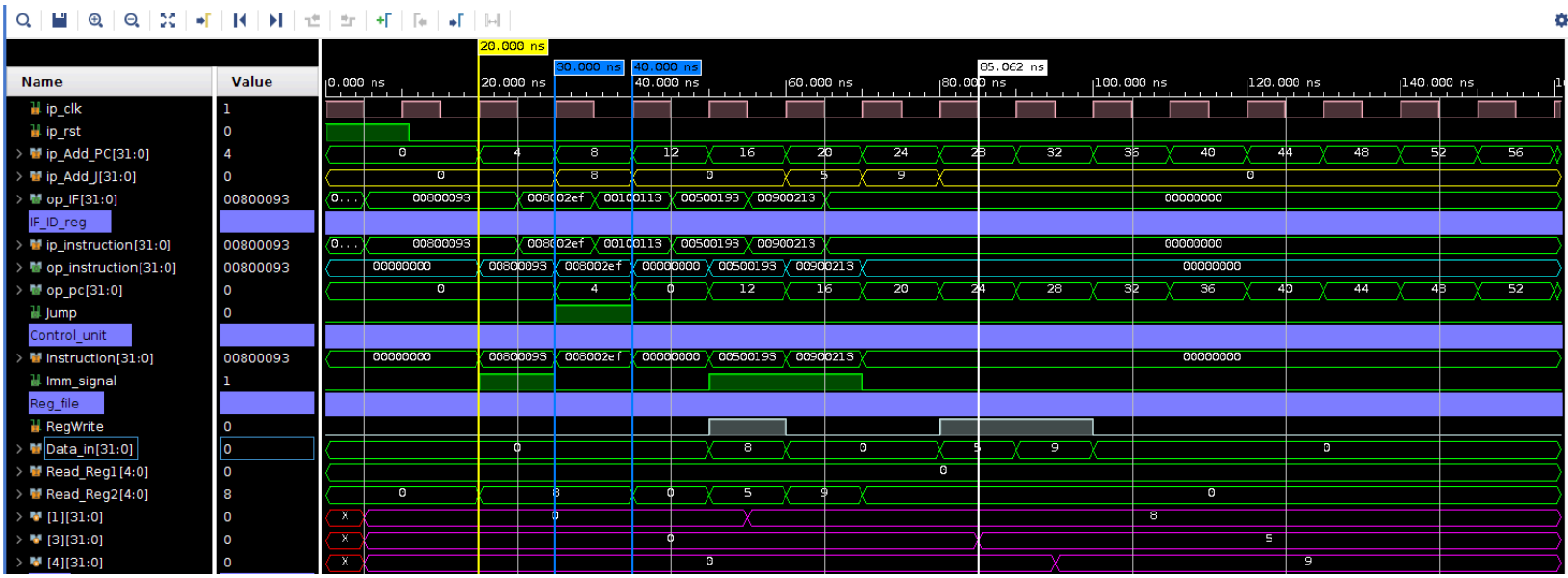
Instruction:

```
addi x1, x0, 8      # x1 = 8 (target address)
jal  x5, 8          # jump to PC+8, save return addr in x5
addi x2, x0, 1      # skipped
addi x3, x0, 5      # executed after jump
addi x4, x0, 9      # next instruction
```

Machine Code:

00800093,
008002EF,
00100113,
00500193,
00900213;

Observation:



Hex Instruction	Assembly Instruction	Meaning
00800093	`addi x1, x0, 8`	x1 = 0 + 8
008002ef	`jal x5, 8`	Jump to PC+8 and store return address in x5
00100113	`addi x2, x0, 1`	x2 = 1
00500193	`addi x3, x0, 5`	x3 = 5
00900213	`addi x4, x0, 9`	x4 = 9

Instruction	Operation	Result	Cycle (approx from waveform)
ADDI	x1 = x0 + 8	x1 = 8	20 ns
JAL	x5 = PC+4, PC = PC + 8	Jump occurs	30 ns
ADDI	x2 = x0 + 1	x2 = 1	40 ns
ADDI	x3 = x0 + 5	x3 = 5	60 ns
ADDI	x4 = x0 + 9	x4 = 9	80 ns

IMMEDIATE INSTRUCTION:

LUI

Instruction

auipc x6, 0x00000
auipc x7, 0x00100

Machine code:
00000317
00100397

Instruction (Hex)	Assembly	Operation	Result
123452b7	`lui x5,0x12345`	x5 = imm << 12	0x12345000
00000317	`auipc x6,0`	x6 = PC + 0	PC value
00100397	`auipc x7,1`	x7 = PC + 4096	PC + 4096

Implementation Results:

Tcl Console | Messages | Log | Reports | Design Runs

Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAM	URAM	DSP
synth_1 (active)	**constrs_1**	**Synthesis Out-of-date**								**1910**	**1544**	**0.0**	**0**	**0**
impl_1	constrs_1	Implementation Out-of-date	0.162	0.000	0.115	0.000	0.000	0.129	0	1913	1544	1.5	0	0
Out-of-Context Module Runs														
IM		Submodule Runs Out-of-date												
d_m		Submodule Runs Complete												

Power:

Tcl Console | Messages | Log | Reports | Design Runs | Power | DRC | Methodology | Timing

Settings
Summary (0.129 W, Margin: N/A)
Power Supply
Utilization Details
Hierarchical (0.047 W)
Clocks (0.004 W)
Signals (0.016 W)
Data (0.016 W)
Clock Enable (<0.001 W)
Set/Reset (<0.001 W)
Logic (0.01 W)
BRAM (0.002 W)
I/O (0.015 W)

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.129 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**25.2°C**
Thermal Margin:	59.8°C (31.5 W)
Effective ϑJA:	1.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium

Launch Power Constraint Advisor to find and fix invalid switching activity

On-Chip Power

37% Dynamic: 0.047 W (37%)
63% Device Static: 0.081 W (63%)

8% Clocks: 0.004 W (8%)
34% Signals: 0.016 W (34%)
22% Logic: 0.010 W (22%)
4% BRAM: 0.002 W (4%)
32% I/O: 0.015 W (32%)

Architecture	Frequency	LUT	FF	Power	BRAM
RISC_V Pipelined	50 MHz	1913	1544	0.129 W	1.5